

2.2 - GENERATIVE AI AND DIFFUSION MODELS

CHAPTER INDEX

- ☐ [GENERATIVE AI](#)
- ☐ [GENERATIVE ADVERSARIAL NETWORK](#)
 - [REFRAMING THE PROBLEM](#)
 - [INPUT SEED AND DIRECT MATCHING](#)
 - [INDIRECT TRAINING](#)
 - [SOME MATHEMATICAL DETAILS](#)
- ☐ [DIFFUSION MODELS](#)
 - [FORWARD DIFFUSION](#)
 - [REVERSE DIFFUSION](#)
 - [LOSS FUNCTION](#)
 - [TRAINING](#)
 - [SAMPLING AND FINAL DETAILS](#)

GENERATIVE AI

Generative models are a class of machine learning models that can **generate new data** that is **similar** to what they **were trained on**.

These models are trained to **understand** the **underlying patterns** and **structure** of the data.

There are many types of generative models, such as **generative adversarial networks (GANs)**, discussed in this chapter), *variational autoencoders (VAEs)*, **diffusion models** (discussed further down this chapter), and *normalizing flow models*.

These models can be used to generate a **wide variety** of data, including **images**, **text**, and **audio**.

To understand **how clever** the concepts behind these models are, consider what Yann LeCun said about *generative adversarial networks*:

| "[They are] the **most interesting idea** in the last 10 years in Machine Learning"

Obviously, such a compliment coming from a prominent researcher in the area is always great advertisement and, indeed, GANs have had a huge success since their introduction in 2014 by **Ian J. Goodfellow** and his co-authors.

The ability to generate new content might make generative models look a little bit like "magic" at first sight, but hopefully the following paragraphs will help overcome this first impression and give helpful insights on the fundamentals behind these models.

⚠ GENERATIVE AI, DEEPPKES AND ETHICS

The use of generative AI to create **deepfakes** has **increased significantly** since the release of these models.

Deepfakes can be used for entertainment, but also to **spread misinformation**, create **fake news**, or **manipulate public perception**, posing significant **ethical** and **security risks**.

Because the capabilities of these models allows for the **rapid** and **widespread** dissemination of **misinformation**, it is becoming **increasingly challenging** for individuals to **identify false narratives**.

Many people are **growing skeptical** of the authenticity of the information they encounter, leading to a **fragmented** and **polarised society**.

GENERATIVE ADVERSARIAL NETWORK

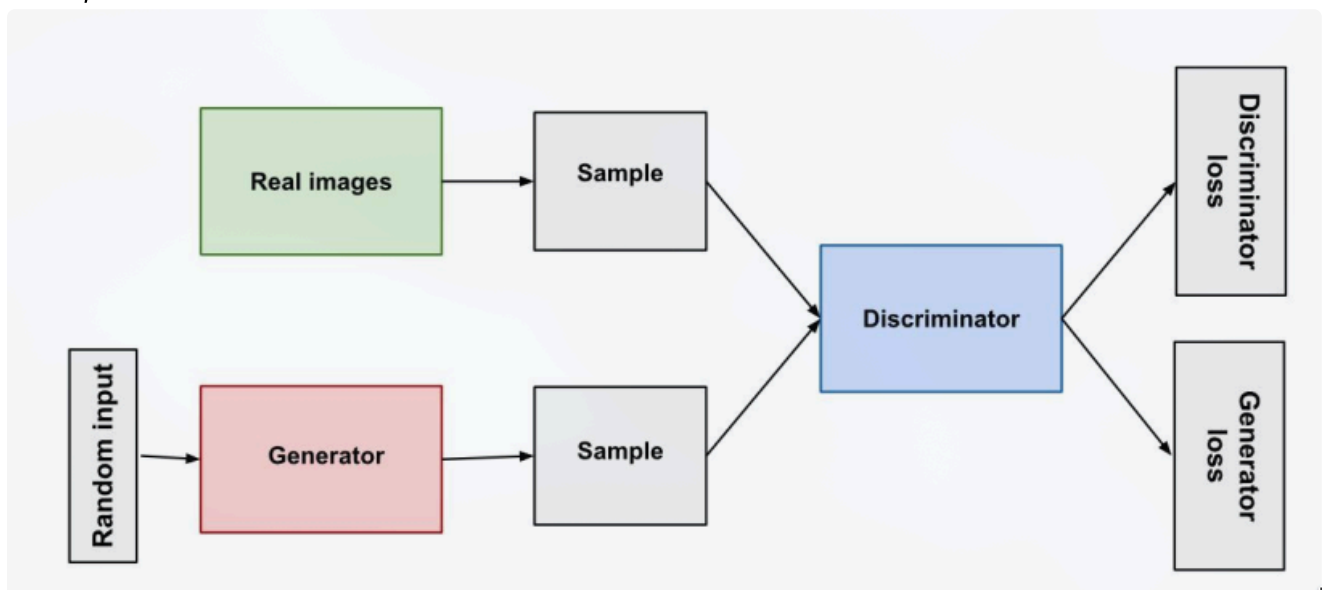
Let us now dive into the architecture and ideas that make GANs possible.

📖 GENERATIVE ADVERSARIAL NETWORK

A **generative adversarial network** is composed of **two parts**:

- A **generator**, which learns to **generate plausible data**. The generated instances become *negative examples* for the *other part*.
- A **discriminator**, which learns to **distinguish** the generator's **fake data** from **real data**. The discriminator **penalises the generator** for producing **implausible results**.

Thus, a GAN can be schematised as shown below:



Both the *generator* and the *discriminator* are **neural networks**.

The **generator output** is **connected directly** to the **discriminator input**. Through *backpropagation*, the **discriminator's classification** provides a **signal** that the **generator** uses to **update its weights**.

NOTES

1. In most cases, the **discriminator** only serves as a means to **optimise the generator**, which is the **network we are interested in training**. Thus, the *discriminator is usually ignored after training ends*.
2. The **generator** and **discriminator** networks are **often CNNs** (although some other architectures may also be used).

REFRAMING THE PROBLEM

Now that we have discussed the *general idea* behind GANs, let us dive into **how exactly they work**.

Suppose that we are interested in **generating** black and white square **images of dogs** of size $n \times n$ pixels.

We can **reshape** each image as an $N = n^2$ dimensional **vector** (e.g. by concatenating rows/columns one after another).

Clearly, not all vectors represent a dog once shaped back to a square matrix: we can **hypothesise** that the N -dimensional **vectors** that effectively **encode** images of **dogs** are **distributed according** to some **very specific probability distribution** over the N -dimensional **vector space**.

GENERALISING THE HYPOTHESIS

It thus seems reasonable to hypothesise that, in general, there **exist probability distribution functions** for **images of all other classes** of objects (e.g. cats, birds, etc.) over this **vector space**.

Then, the problem of **generating new images of dogs** is **equivalent** to the problem of **generating vectors** that **follow the "dog probability distribution"** (at least in our example case, the class will of course differ in each application).

The **tricky part** is that, obviously, we **do not know** how to explicitly **express this distribution**.

INPUT SEED AND DIRECT MATCHING

To understand **how to achieve** the **desired output distribution**, it is important to **first think about what kind of input** our network expects.

Normally, a network's input is *data* that we want to do something with, like an instance that we want to *classify* or *make a prediction* about.

But what do we use as input for a network that **outputs entirely new data instances**?

✍ RANDOM NOISE INPUT

In its most **basic form**, a GAN takes **random noise** as its input.

The **generator** then transforms this noise into a **meaningful output**.

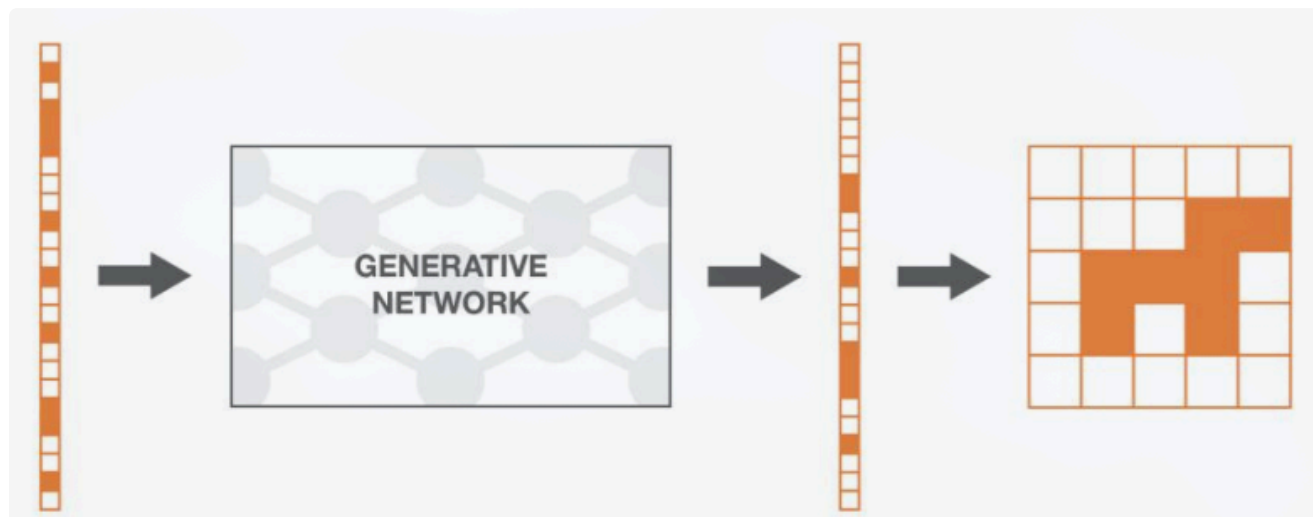
Therefore, our generator essentially needs to **learn a transformation** that maps a **source of noise** to **structured data**.

💡 INTERPRETING THE INPUT

We can think of the input as a **seed** that the network uses to **produce its output**.

Because this seed is **noisy**, we can get the GAN to produce a **wide variety of data**: we are essentially using noise to **sample** from **different parts** of the **target distribution**.

The figure below illustrates this concept:



Experiments suggest that the **distribution of the noise doesn't matter much**, so we can choose one that is easy to sample from, like a **uniform distribution**.

Furthermore, for convenience, the space from which the **noise is sampled** is often of **smaller dimension** than the dimensionality of the **output space**.

Before diving into how to optimise the generator with the help of a discriminator, it may be useful to first consider a **simpler approach: Generative Matching Network (GMNs)**. The idea behind these models is to **train them by comparing the generated distribution** to the **true one**.

COMPARING THE DISTRIBUTIONS

As already stated above, we **do not know** how to explicitly express the **target distribution**, and the **generated distribution** might too be **excessively complex**.

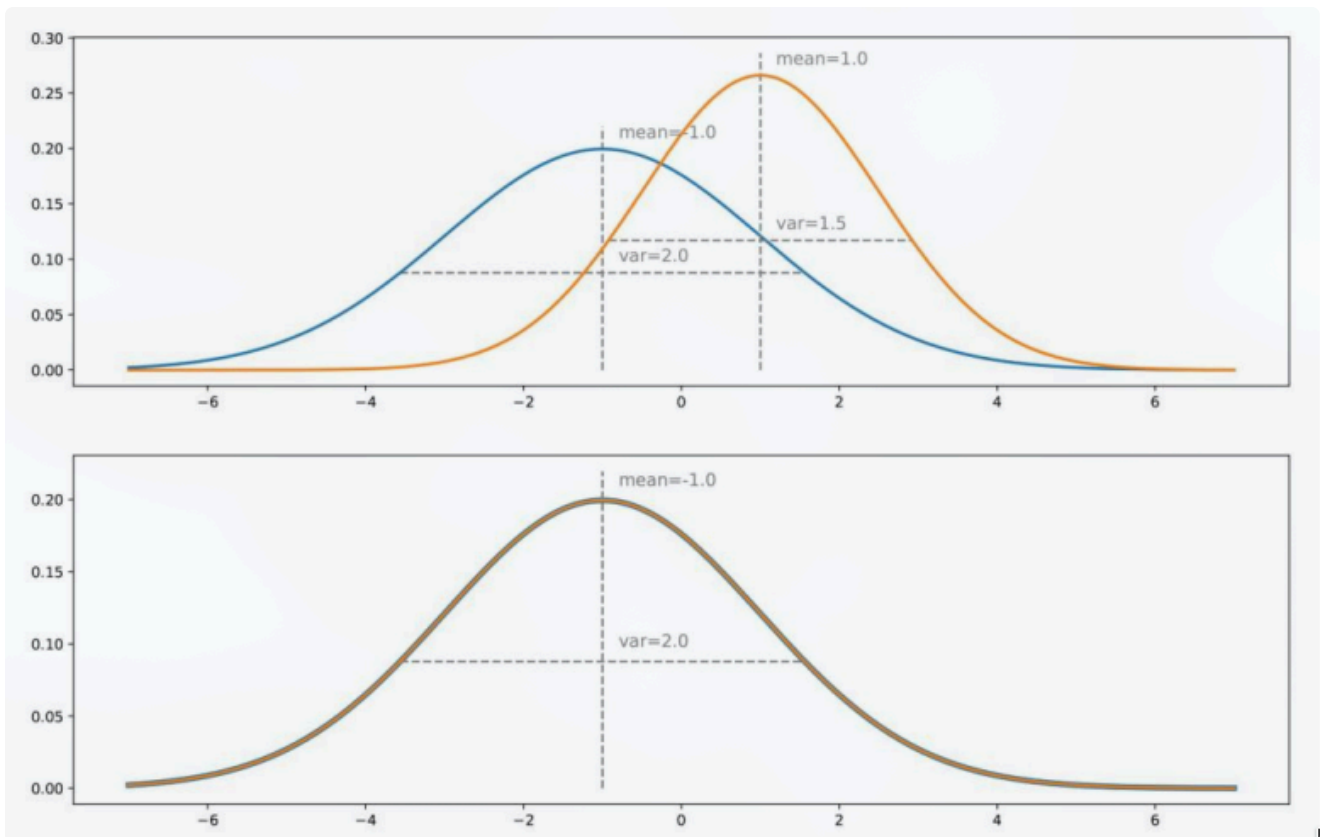
However, methods to **compare distributions based on samples** exist, and these can indeed be used, as both *true data* and *generated data* samples are available.

Therefore, we can train a GMN by repeating the following steps:

TRAINING A GMN

1. Generate some **uniform inputs**.
2. Perform **forward propagation** through the network to obtain outputs.
3. **Compare** the **true distribution** and the **generated distribution** based on the available **samples** to calculate **loss**.
4. Use **backpropagation** to compute the gradients and **update** the model's **parameters**.

Notice that this is the same process we have already discussed for other networks in the previous sections; the only difference stems from the fact that the "**loss function**" here is a **measure of distance between two probability distributions** (e.g. Maximum Mean Discrepancy).



DIRECT MATCHING

As shown in the figure above, at the **end of the training phase** (assuming a perfect process) the **generated distribution matches the true one**; this approach is thus also known as **direct matching**.

INDIRECT TRAINING

The **direct approach** presented above **compares directly** the *generated distribution* to the *target distribution* during **training** using methods that **measure the distance** between the **two distributions**.

This introduces a problem, related to the **curse of dimensionality**.

ISSUE WITH DISTANCE MEASURES

When working with **high-dimensional spaces**, methods that measure the distance between probability distributions **fail to be accurate**.

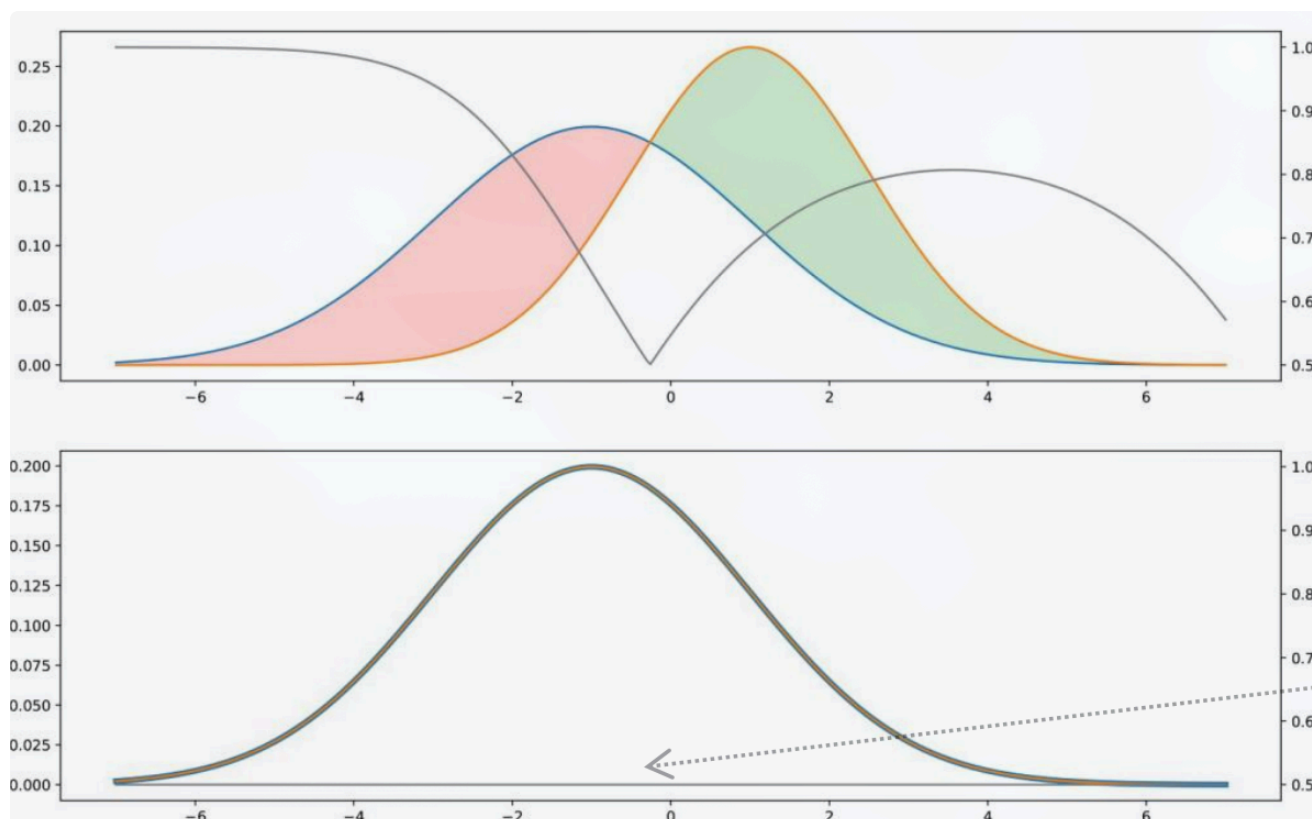
Intuitively, this is because the **volume of the space grows so fast** (and thus so do distances) that **distance fails to capture meaningful information**.

This issue can be addressed by **replacing** the **direct matching** process with an **indirect approach**, with the supervision of a **discriminator**.

During training, the **discriminator "judges"** the **generator's output**, forcing it to **produce more plausible outputs** that will in turn more easily **fool the discriminator**.

At the end, this process will too lead the **generated distribution** to **match the target** one, just like in the direct matching approach.

To convince ourselves of this equivalence of the two approaches, consider the figure below:



In the figure above, we can see the **target distribution** in *blue*, the **generated distribution** in *orange*, and the **probability** that the **discriminator correctly distinguishes fake and real data** (computed as if it were to choose the distribution with higher density) in *grey*.

🧐 "FOOLING" THE DISCRIMINATOR

We can see that the **discriminator's output** is **more likely to be correct** when the two **distributions** take **different values**.

Similarly, it becomes **less confident** when the **distributions match** (around 0 in the top situation, and over the entire range of values in the bottom case).

SOME MATHEMATICAL DETAILS

Mathematically, GANs can be modelled with a **game theoretic approach** as follows.

To start, we define the **noise source** distribution as $p_z(z)$ (as stated above, this is often **uniform**) and the **real data source** as $p_r(x)$.

To learn the **generator's distribution** p_g over **data** x , we represent the **mapping** to the **data space** as $G(z; \theta_g)$, where G is a differentiable function represented by a **multilayer perceptron** with **parameters** θ_g .

We also define a **second MLP** $D(x; \theta_d)$ that **outputs a single scalar** $D(x)$ which

represents the **probability** that x came from the **real data** rather than p_g (that is, the probability that x is "*genuine*").

With the given definitions, the **training process** of a GAN can be formulated as a **minimax game**:

📖 GAN GAME

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_r(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Notice that the above **loss function** derives from **Binary Cross-Entropy Loss**, though the sign is reversed for the sake of simplicity.

💡 OBSERVATIONS

1. The **first term** deals with inputs sampled from the **real data**, while the second term accounts for the **generated data** fed to the discriminator.
2. Notice that if G generates a **perfect fake** we will have $D(G(z)) \approx 1$, and thus $\log(1 - D(G(z))) \rightarrow -\infty$. As a result, we want to **minimise** V with respect to the **generator** (because making it very *negative* means it successfully *fooled the discriminator*). Conversely, we want to **maximise** V with respect to the **discriminator**: in fact, the two terms take their maximum values (0) when the **data is real** ($\log D(x) = \log(1) = 0$) and when **a fake input is correctly spotted** ($\log(1 - D(G(z))) = \log(1 - 0) = 0$).

Now suppose that, for a **fixed generator** G , we want to find the **optimal discriminator** D ; that is, we want to find:

$$\max_D V(D, G) = \mathbb{E}_{x \sim p_r(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

The second expectation is *over noise drawn* from p_z and $G(z)$ ultimately produces a **fake sample** according to p_g . But p_r and p_g have the **same support** $\mathcal{X}$, therefore we can rewrite the second expectation as:

$$\mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] = \mathbb{E}_{x \sim p_g} [\log(1 - D(x))]$$

Since both distributions are continuous, we can rewrite $V(D, G)$ as follows:

$$V(D, G) = \int_{\mathcal{X}} p_r(x) \log D(x) + p_g(x) \log(1 - D(x)) dx$$

Now we want to **maximise this integral**. Because there are no constraints linking $D(x_1)$ with $D(x_2)$ for any $x_1, x_2 \in \mathcal{X}$, we can **equivalently maximise the integrand**.

For simplicity, we define a **helper function** $f(D)$ for a **fixed** x :

$$f(D) = a \log(D) + b \log(1 - D)$$

Notice that we have $a = p_r(x)$ and $b = p_g(x)$, both *constants* for a *specific value* of x . To maximise D , we take its derivative and set it to zero:

$$\frac{df(D)}{dD} = \frac{a}{D} - \frac{b}{1-D}$$

$$\frac{df(D)}{dD} = 0 \implies D^* = \frac{a}{a+b}$$

And we obtain D^* by substituting a and b :

$$D^*(x) = \frac{p_r(x)}{p_r(x) + p_g(x)}$$

This is indeed a **maximum**, as $f''(D) < 0$ for all x .

🔗 IMPLICATION

We immediately notice that, if $p_g(\hat{x}) = p_r(\hat{x})$ for some $\hat{x} \in \mathcal{X}$ (that is, a specific image is **equally likely** to appear in the **target distribution AND** in the **generated distribution**), then $D^*(\hat{x}) = 0.5$: the **discriminator** is **completely uncertain** about the origin of $\hat{x}$.

Consequently, if $p_g(x) = p_r(x) \ \forall x \in \mathcal{X}$ it means we have managed to build the **perfect generator**.

Notice that we substantially built a solid **mathematical basis** for the **intuitions** that we had seen earlier when discussing **indirect training**.

DIFFUSION MODELS

Another **approach** to **generative AI** is given by **diffusion models**, which are highly employed to generate **high quality images and videos** (a famous diffusion model is Midjourney).

Both GANs and diffusion models **start with a prior (noise)** and **learn a mapping** to some **complex data space**.

However, they differ in how the way they tackle the problem: unlike GANs, which required the "*supervision*" of a discriminator; diffusion models **break the transformation** into a **multitude of tiny, incremental steps** (hence the name, which resembles natural diffusion processes in physics).

🔗 INTUITION

The intuition behind this approach is that the model can **correct itself** over these **small steps** and **gradually produce** a good sample.

In particular, diffusion models work in a **dual-phase mechanism**:

1. First, we **introduce noise into the dataset**.
2. Then, we **train a network** to methodically **reverse this process**.

NOTE

Before beginning the diffusion process, data needs to be properly adjusted for the training phase.

This involves **removing outliers**, **data normalisation** to *scale features consistently*, and **data augmentation** to *increase dataset diversity* (see > [1.1 - MORE ON NETWORK TRAINING > DATA AUGMENTATION](#)).

Furthermore, **different data types** may require **specific preprocessing steps**.

FORWARD DIFFUSION

The goal of the first phase is to **add noise** to the **dataset**: this is done with a process known as **forward diffusion**.

Mathematically, given a **starting data-point** $\mathbf{x}_0$ sampled from the **real data** distribution $q(\mathbf{x})$ (that is, $\mathbf{x}_0 \sim q(\mathbf{x})$), we define a **forward diffusion process** of T **steps** as follows:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t I), t = 1, \dots, T$$

Where each $q(\mathbf{x}_t|\mathbf{x}_{t-1})$ is a **multivariate distribution** with **means** given by the vector $\mu_t = \sqrt{1 - \beta_t}\mathbf{x}_{t-1}$ and **covariance matrix** $\Sigma_t = \beta_t I$ (notice that this matrix is diagonal: all **variables** are **independent**).

CLARIFICATION

Here, the **data points** $\mathbf{x}$ and their **distribution** $q(\mathbf{x})$ are treated as **vectors**.

Furthermore, though diffusion models can be used to generate various kinds of data, we will present them in the context of **image generation** for the sake of clarity.

The complete forward process can be expressed as follows:

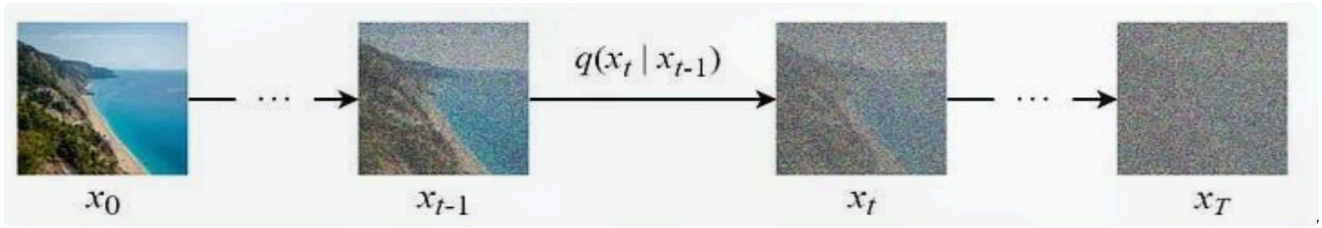
$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})$$

OBSERVATION

At **each step**, we are producing a **new latent variable** $\mathbf{x}_t$ with distribution $q(\mathbf{x}_t|\mathbf{x}_{t-1})$ by **adding Gaussian noise** with **variance** β_t to the **previous variable**

$\mathbf{x}_{t-1}$.

The result is a **series of increasingly noisy samples**.



Notice that we used "latent variable" to describe each $\mathbf{x}_t$; this is because we are referring to a **hidden, continuous feature space** (do not confuse this with the concept of autoencoders seen in > [2.0 - CONVOLUTIONAL NEURAL NETWORKS > AUTO ENCODER](#)).

NOTE

The value of β_t is a **hyperparameter**.

Furthermore, this can either be a **fixed value** $\forall t$, or it can **vary based on a schedule** over the T timesteps; this schedule can be *linear, quadratic, cosine* etc.

The original **authors** utilised a **linear schedule** increasing from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$.

From the definition above, it may look like we need to **compute every intermediate step** to reach the result at noise level T .

In reality, we can **efficiently compute** the result of this diffusion process, without having to go through each intermediate calculation.

To start, recall that we can **sample values** from a normal distribution $\mathcal{N}(\mu, \sigma^2)$ as follows:

$$x = \mu + \sigma\epsilon$$

With $\epsilon \sim \mathcal{N}(0, 1)$.

Thus, for $t = 1, \dots, T$ we have:

$$\mathbf{x}_t = \sqrt{1 - \beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\epsilon_{t-1}$$

We now define:

$$\alpha_t = 1 - \beta_t \quad \bar{\alpha}_t = \prod_{s=0}^t \alpha_s$$

So we can rewrite:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon_t$$

By substituting the expression for $\mathbf{x}_{t-1}$ we obtain:

$$\begin{aligned}
\mathbf{x}_t &= \sqrt{\alpha_t}(\sqrt{\alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_t}\epsilon_{t-2}) + \sqrt{1 - \alpha_t}\epsilon_{t-1} \\
&= \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \underbrace{\sqrt{\alpha_t(1 - \alpha_{t-1})}\epsilon_{t-2}}_X + \underbrace{\sqrt{1 - \alpha_t}\epsilon_{t-1}}_Y
\end{aligned}$$

We notice that:

$$X \sim \mathcal{N}(0, \alpha_t(1 - \alpha_{t-1})I)$$

$$Y \sim \mathcal{N}(0, (1 - \alpha_t)I)$$

We thus have:

$$\begin{aligned}
Z = X + Y &\sim \mathcal{N}(0, [\alpha_t(1 - \alpha_{t-1}) + (1 - \alpha_t)]I) \\
&= \mathcal{N}(0, 1 - \alpha_t\alpha_{t-1})
\end{aligned}$$

So it follows:

$$Z = 0 + \sqrt{1 - \alpha_t\alpha_{t-1}}\epsilon_{t-2}$$

Finally, we obtain:

$$\mathbf{x}_t = \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_t\alpha_{t-1}}\epsilon_{t-2}$$

🔍 OBSERVATION

We can **keep repeating this process** to reach a **closed form** expression for $\mathbf{x}_t$.

This closed form expression is:

$$\begin{aligned}
\mathbf{x}_t &= \sqrt{\alpha_t\alpha_{t-1}\dots\alpha_1}x_0 + \sqrt{1 - \alpha_t\alpha_{t-1}\dots\alpha_1}\epsilon \\
&= \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon
\end{aligned}$$

📝 NOTE

What we showed above is known as **reparametrisation trick**, and it allowed us to obtain a closed form expression which we can use to **directly compute any $\mathbf{x}_t$** without having to go through the entire process.

REVERSE DIFFUSION

The **forward step** let us go from the original image $\mathbf{x}_0$ **up to a complete noise image**. What we really **want to know**, though, is the **reverse process**; that is, how we can go **from noise to an image**.

This process can be described in a **similar way**, but in the **opposite direction**:

$$p(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

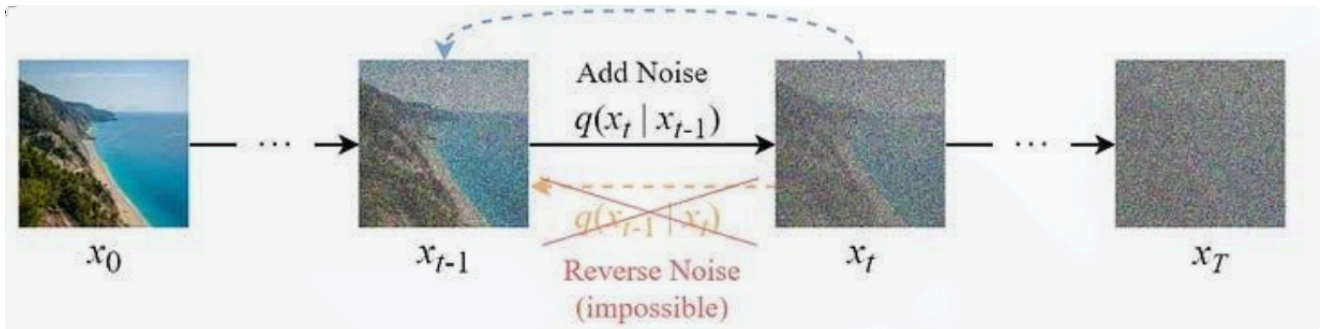
Where $p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; 0, I)$; that is, the process should ideally **start** from an **isotropic Gaussian noise image** (meaning the noise follows the same distribution along *all possible directions* starting from the origin).

Notice that reversing the forward process requires finding the "**opposite**" **conditional distribution** $p(\mathbf{x}_{t-1}|\mathbf{x}_t)$.

💡 MOTIVATION

This would let us **gradually remove the noise** until we end up with an **image** from the **real data distribution** $\mathbf{x}_0$.

The problem, of course, is that we do **not know this distribution**: thus, we try to **estimate it** using a **neural network**.



For this estimation, we replace the original unknown distribution $p(\mathbf{x}_{t-1}|\mathbf{x}_t)$ with a **parametric** one, as follows:

$$p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

Where θ denotes the **network's parameters**.

Because we are dealing with **Gaussian distributions**, $p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)$ can be fully **described** by a **mean** and a **variance**, and so we can parameterise it as:

$$p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_{\theta}(\mathbf{x}_t, t), \Sigma_{\theta}(\mathbf{x}_t, t)), t = 1, \dots, T - 1$$

With $p_{\theta}(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; 0, I)$.

✍ ASSUMPTION

We can make an **assumption** for our model (and **make it simpler**) and decide that the **variance is not a parameter** the network needs to learn (this same assumption was made in the original paper, but note that later works decided that the model should also learn the variance).

As we will see below, this **greatly simplifies** the **derivation** of the **loss function**.

We thus set $\Sigma_\theta(\mathbf{x}_t, t) := \beta_t I$ for some β_t that **doesn't depend** on θ .

LOSS FUNCTION

So far we have discussed **how** the **diffusion** processes **work**, but currently we **do not** even have a **precise idea** of **what** our **network** should be **predicting**.

🔗 FIRST INTUITION

We said in the introduction that we want to **train the network** to **reverse the diffusion process**: this leads us to assume that the network's **output** might be the **noise** that we need to **remove from the input** to obtain a **real image**.

It turns out that this is indeed true, but let us give a rigorous mathematical argument as to why.

First of all, assume that the **distribution of images generated** by our network after the full **reverse process** is given by $p_\theta(\mathbf{x}_0)$.

Instinctively, we might think of taking the **expected log likelihood** over $\mathbf{x}_0 \sim q(\mathbf{x})$ of this distribution and **maximising it**; that is, since $\mathbf{x}_0$ is a **valid image**, as it is sampled from the target distribution, we want our **network's probability** of obtaining it to be **high**. Because we are in a deep learning context, we prefer **minimising the negative log likelihood**, but there is a catch.

The **problem** with this approach is that, to estimate the distribution of $\mathbf{x}_0$ in the reverse process, we need to **keep track of all the T steps before**.

Instead, we can find **another function** that we **can optimise** to act as an **upper bound** for it.

Thus, we may consider the following inequality:

$$\begin{aligned} -\log p_\theta(\mathbf{x}_0) &\leq -\log p_\theta(\mathbf{x}_0) + D_{\text{KL}}(q(\mathbf{x}_{1:T}|\mathbf{x}_0) || p_\theta(\mathbf{x}_{1:T}|\mathbf{x}_0)) \\ &= -\log p_\theta(\mathbf{x}_0) + \mathbb{E}_{\mathbf{x}_{1:T} \sim q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] \end{aligned}$$

Notice that we added the **KL Divergence** (recall that we have discussed entropy and KL divergence in > [1.0 - NEURAL NETWORKS INTRODUCTION > CROSS-ENTROPY LOSS](#)) between the **forward process** and the **reverse process** to the right-hand side, so that it is always greater than or equal to our log likelihood.

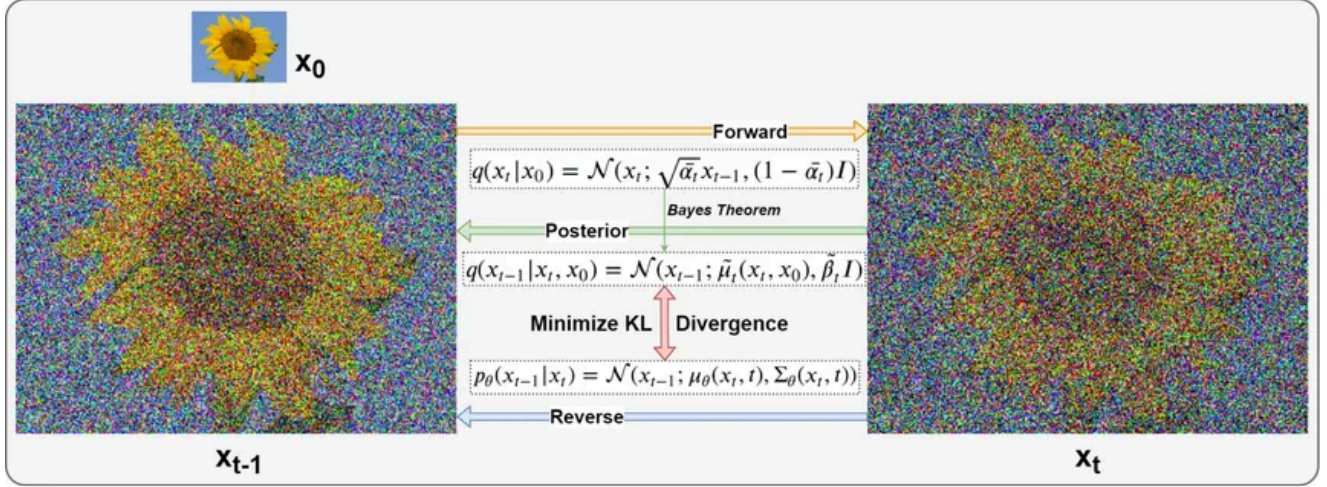
Indeed, **this function can be optimised**.

Notice that choosing to **minimise this other function** has an interesting implication:

💡 FURTHER INSIGHT

By **minimising the KL divergence** between the **two processes**, we are effectively **forcing the network to learn a trajectory that matches the forward process** as closely as possible, thus **accurately reversing it**.

This intuition may be better understood by looking at the image below:



Now let us show why this addition is also *mathematically* convenient.

First, we apply Bayes' rule to the numerator of the log's argument:

$$p_\theta(\mathbf{x}_{1:T}|\mathbf{x}_0) = \frac{p_\theta(\mathbf{x}_0|\mathbf{x}_{1:T})p_\theta(\mathbf{x}_{1:T})}{p_\theta(\mathbf{x}_0)} = \frac{p_\theta(\mathbf{x}_0, \mathbf{x}_{1:T})}{p_\theta(\mathbf{x}_0)} = \frac{p_\theta(\mathbf{x}_{0:T})}{p_\theta(\mathbf{x}_0)}$$

Substituting this back into the logarithm gives us:

$$\log \left(\frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right) = \log \left(\frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{\frac{p_\theta(\mathbf{x}_{0:T})}{p_\theta(\mathbf{x}_0)}} \right) = \log \left(\frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} \right) + \log p_\theta(\mathbf{x}_0)$$

Finally, substituting this into the original inequality and taking the expectation over $\mathbf{x}_0 \sim q(\mathbf{x})$ we obtain:

$$E_q[-\log p_\theta(\mathbf{x}_0)] \leq E_q \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} \right]$$

✍ NOTE

We use E_q to denote the expectation over $\mathbf{x}_0 \sim q(\mathbf{x})$ and $\mathbf{x}_{1:T} \sim q(\mathbf{x}_{1:T}|\mathbf{x}_0)$ for convenience, to avoid cluttering equations with symbols.

Notice that the distributions inside the logarithm are the **forward process** and **reverse process** we defined in the paragraphs above. We thus have:

$$\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} = \log \frac{\prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} = -\log p_\theta(\mathbf{x}_T) + \sum_{t=1}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)}$$

Where we simply used properties of logarithms to obtain the last result.

Now we rewrite the above expression as:

$$-\log p_\theta(\mathbf{x}_T) + \sum_{t=1}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} = -\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)}$$

With some (tedious) manipulations involving Bayes' rule, we can further manipulate the above result to write it in a convenient form.

✍ NOTE: SKIPPED CALCULATIONS

In short, we apply Bayes' rule to the *forward process* terms and *condition* each step on the *input image* $\mathbf{x}_0$. Then, by using properties of logarithms we see that some *convenient cancellations are possible*.

Finally, with some more algebraic manipulation, we can *rewrite* the terms in such a way that makes *KL divergence appear*.

$$\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} = \underbrace{D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0)||p_\theta(\mathbf{x}_T))}_{L_T} + \left(\sum_{t=2}^T \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)||p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_t} \right) \underbrace{-\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{L_0}$$

Finally, this is the **function** that we **can optimise**.

Remember that this represents the **loss** for a **single training example**; the **true training loss** is computed by taking the **expectation** of the above expression over $\mathbf{x}_0 \sim q(\mathbf{x})$.

Now, we will look at each term individually and omit the expectation for convenience.

First of all, let us look at L_T .

🔍 OBSERVATION: DROPPING THE FIRST TERM

We conveniently notice that $q(\mathbf{x}_T|\mathbf{x}_0)$ **does not depend** on the **model's parameters** θ (the forward process is *not performed by the network*), and nor does $p_\theta(\mathbf{x}_T)$ (recall that it was defined as $\mathcal{N}(\mathbf{x}_T; 0, I)$).

As a result, we **can drop it entirely** as it **doesn't affect the gradients** during backpropagation.

We can thus move on to the L_t terms (for $t = 2, \dots, T$). Recall that we have:

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \beta_t I)$$

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\mu}(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t I)$$

Because we don't know how to write $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ explicitly, we use Bayes' rule to obtain:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0)q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)}$$

Now, we **know** how to express **all three distributions** on the **right hand side** of the equation thanks to the **reparametrisation trick** we presented earlier.

By expressing these in exponential form, and plugging them back into the equation for $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$, we obtain:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \propto \exp \left(-\frac{1}{2} \left[\frac{(\mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_{t-1})^2}{\beta_t} + \frac{(\mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0)^2}{1 - \bar{\alpha}_{t-1}} - \frac{(\mathbf{x}_t - \sqrt{\bar{\alpha}_t} \mathbf{x}_0)^2}{1 - \bar{\alpha}_t} \right] \right)$$

Now, we want to rewrite this expression in the following form:

$$\mathcal{N}(x; \mu, \sigma^2) \propto \exp \left(-\frac{1}{2} \frac{(x - \mu)^2}{\sigma^2} \right) = \exp \left(-\frac{1}{2} \left[\frac{1}{\sigma^2} x^2 - \frac{2\mu}{\sigma^2} x + \frac{\mu^2}{\sigma^2} \right] \right)$$

With some algebra, we obtain:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \propto \exp \left(-\frac{1}{2} \left[\underbrace{\left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}} \right)}_{1/\sigma^2} \mathbf{x}_{t-1}^2 - \underbrace{\left(\frac{2\sqrt{\alpha_t}}{\beta_t} \mathbf{x}_t + \frac{2\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}} \mathbf{x}_0 \right)}_{2\mu/\sigma^2} \mathbf{x}_{t-1} + \underbrace{f(\mathbf{x}_t, \mathbf{x}_0)}_{\mu^2/\sigma^2} \right] \right)$$

This **allows** us to **find the mean and variance** of this distribution (notice that the last term has been written as a generic function f , as we don't need it for this calculation) by comparing each underbraced term with the corresponding coefficient.

We thus find:

$$\sigma^2 = \tilde{\beta}_t = 1 - \frac{\bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$$

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0$$

From the expression for $\mathbf{x}_t$ we can isolate $\mathbf{x}_0$:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon \implies \mathbf{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} (\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \epsilon)$$

By substituting this into the previous equation (and with some algebra) we obtain the following expression for $\tilde{\mu}_t$:

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right)$$

Ideally, we want our network to **learn** its μ_θ to **predict the target** $\tilde{\mu}_t$.

We may thus set:

$$\mu_{\theta}(\mathbf{x}_t, t) := \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right)$$

🔗 NETWORK'S PREDICTION

Notice that $\epsilon_{\theta}(\mathbf{x}_t, t)$ is the **only prediction** that our **network can make**, as *every other parameter is fixed*.

Indeed, this **confirms our initial assumption** that the network's **output** is the **noise that was added**: that is, if we take the noisy sample and **subtract the network's prediction** from it, we should get the **original image back**.

Now, going back to the loss function derivation, we have **everything necessary** to write the **first piece of the loss function**.

🔗 LOSS FUNCTION

Because the **only parameter** that depends on θ (thanks to the assumption in the original paper that the model won't need to learn the variance) is the **mean**, it can be shown that the **KL divergence** can be **simplified** into **MSE**.

We thus have:

$$L_t = \frac{1}{2\sigma_t^2} \|\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}) - \mu_{\theta}(\mathbf{x}_t, t)\|^2$$

Plugging the expressions found earlier we obtain:

$$L_t = \frac{(1 - \alpha_t)^2}{2\alpha_t(1 - \bar{\alpha}_t)\sigma_t^2} \|\epsilon - \epsilon_{\theta}(\mathbf{x}_t, t)\|^2$$

🔗 NOTE: FURTHER SIMPLIFICATION

In the **original paper**, authors found empirically that the training works better if the **scaling factor is omitted**.

Thus, we can simplify this loss to:

$$L_t = \|\epsilon - \epsilon_{\theta}(\mathbf{x}_t, t)\|^2$$

To conclude the derivation, we need to **handle the last term** of our loss:

$$L_0 = -\log p_{\theta}(\mathbf{x}_0 | \mathbf{x}_1).$$

This is the **reconstruction term** and it measures how well the model can **recover the original image** from the **first noisy step** $\mathbf{x}_1$.

One question that could arise when looking at L_0 is the following:

Why should we treat it separately? What is so special about L_0 compared to the other L_t terms?

L_0 MUST BE HANDLED DIFFERENTLY

The answer lies in the fact that, while our **diffusion** process treats images as **continuous vectors** in $\mathbb{R}^D$ (where D is the number of pixels), **real images are discrete**; that is, each pixel takes an *integer value* in $[0, 255]$.

Thus, when we compute $-\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)$, we need the **probability** that the **model assigns** to the **exact discrete pixel values** of the **original image** $\mathbf{x}_0$; recall that this probability was defined as:

$$p_\theta(\mathbf{x}_0|\mathbf{x}_1) = \mathcal{N}(\mathbf{x}_0; \mu_\theta(\mathbf{x}_1, 1), \beta_1 I)$$

The problem is we **cannot** just **evaluate a continuous Gaussian** PDF at a single point, because its **probability would be zero**; we thus need a way to **discretise it**.

The original paper assumes that the **pixel values** are **scaled** to the **range** $[-1, 1]$, and thus approximates $p_\theta(\mathbf{x}_0|\mathbf{x}_1)$ as follows:

$$p_\theta(\mathbf{x}_0|\mathbf{x}_1) = \prod_{i=1}^D \int_{\delta_-(x_0^i)}^{\delta_+(x_0^i)} \mathcal{N}(x; \mu_\theta^i(\mathbf{x}_1, 1), \beta_1 I) dx$$

Where:

$$\delta_+(x) = \begin{cases} \infty & x = 1 \\ x + \frac{1}{255} & x < 1 \end{cases} \quad \delta_-(x) = \begin{cases} -\infty & x = -1 \\ x - \frac{1}{255} & x > -1 \end{cases}$$

INTERPRETATION

We've seen that our network **predicts the mean of a pixel** (here, μ_θ^i).

We thus use this prediction to **draw a Gaussian** distribution and **integrate over it** in a **tiny range** centred **around the real pixel value** x_0^i in the original image $\mathbf{x}_0$.

Notice that the **value** of this **integral** is **larger** if the **predicted mean** μ_θ^i is **close** to the **true pixel value** x_0^i .

By **repeating** this for **every pixel** x_0^i $i = 1, \dots, D$ in the image and **multiplying the results**, we obtain the probability over the **entire image**.

However, the original paper *further simplified* this loss: instead of calculating the integral, it is **approximated by the Gaussian** probability density function times the **bin width**:

$$p_\theta(\mathbf{x}_0|\mathbf{x}_1) \approx \frac{1}{\sqrt{2\pi\beta_1}} \exp\left(-\frac{1}{2} \frac{\|\mathbf{x}_0 - \mu_\theta(\mathbf{x}_1, 1)\|^2}{\beta_1}\right) \cdot \frac{2}{255}$$

Which leads to the following **loss term**:

$$L_0 = -\log p_\theta(\mathbf{x}_0|\mathbf{x}_1) \approx \frac{1}{2\beta_1} \|\mathbf{x}_0 - \mu_\theta(\mathbf{x}_1, 1)\|^2 + C$$

Both the **constant** and the **scaling factor** can be **ignored**.

Furthermore, we recall that:

$$\mathbf{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_1}}(\mathbf{x}_1 - \sqrt{1 - \bar{\alpha}_1}\epsilon)$$

$$\mu_\theta(\mathbf{x}_1, 1) = \frac{1}{\sqrt{\bar{\alpha}_1}}\left(\mathbf{x}_1 - \frac{1 - \alpha_1}{\sqrt{1 - \bar{\alpha}_1}}\epsilon_\theta(\mathbf{x}_1, 1)\right)$$

Substituting these into L_0 yields:

$$L_0 \approx \|\epsilon - \epsilon_\theta(\mathbf{x}_1, 1)\|^2$$

Now that we have expressions for L_t and L_0 , we can finally write the **loss function** for training.

$$L = \|\epsilon - \epsilon_\theta(\mathbf{x}_1, 1)\|^2 + \sum_{t=2}^T \|\epsilon_t - \epsilon_\theta(\mathbf{x}_t, t)\|^2$$

🔍 OBSERVATION

Because we **omitted the scaling factors** in L_t and L_0 , L turns out to be **particularly simple**.

In fact, since both L_0 and the L_t terms have the **same form**, we can write:

$$L = \sum_{t=1}^T \|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2$$

$$= \sum_{t=1}^T \|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)\|^2$$

📝 NOTE

Notice that the **loss function** presented above is the loss for a **single training example** $\mathbf{x}_0$ that has gone through **forward diffusion processes** of $1, 2, \dots, T$ steps.

Ideally, we want to use **different examples at different noise levels**.

Thus, our loss function should be a slightly modified version of the above expression, that takes **expectation** into consideration:

$$L = \mathbb{E}_{t \sim U[1, T], \mathbf{x}_0 \sim q(\mathbf{x}), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\left\| \epsilon - \epsilon_\theta \left(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t \right) \right\|^2 \right]$$

🔑 KEY OBSERVATION

Notice that we **began the derivation** of the loss function by introducing the **KL divergence** between the forward and reverse **processes**, but we arrived at a result that **only depends on the *entire noise* applied** to the sample.

This may seem confusing, but it is *precisely what we wanted*, because it lets us **efficiently compute the loss without** having to **account for each step**.

TRAINING

Now that we have **derived a loss function** for our **diffusion model**, we can finally discuss how to **train the network**.

The **training algorithm** that was proposed in the original paper is the following:

Algorithm 1 Training

- 1: **repeat**
 - 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
 - 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
 - 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 5: Take gradient descent step on

$$\nabla_\theta \left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$$
 - 6: **until** converged
-

📋 TRAINING DIFFUSION MODELS

1. We **sample an image** $\mathbf{x}_0$ from the data distribution $q(\mathbf{x})$.
2. We **sample a target time-step** t from a **uniform distribution**; this way, we can train the network *evenly* on **different noise levels**.
3. We **sample a Gaussian distribution** to **introduce the noise** into the image.
4. We **feed the noisy image** and the **timestep** to the network. Our model will then make a **prediction** about the **noise that was added**.
5. We **optimise the model weights** by performing **gradient descent** with the **loss function** derived above.
6. **Repeat** 1-5 until convergence is reached.

It is important to notice that the original diffusion model **did not feed the timestep directly** into the network; instead, **sinusoidal positional embedding** was used to produce an **encoding for the time-steps**.

These encodings are **fixed** (there are **no learnable parameters**) and they are computed as follows:

TIMESTEP EMBEDDING

1. **Choose the embedding dimension d** (e.g. 128 in the original paper).
2. For **each even index i** in $[0, d)$, compute:

$$\text{emb}[i] = \sin\left(\frac{t}{10000^{i/d}}\right)$$

$$\text{emb}[i + 1] = \cos\left(\frac{t}{10000^{i/d}}\right)$$

Positional embedding is also used in **transformer** models (see > [3.1 - TRANSFORMERS](#)) and will be discussed in more detail in the dedicated chapter. For now, it may just be useful to consider the following intuition:

INTUITION

By using a **multi-dimensional representation** of the time-steps we are allowing the network to **more easily capture complex relationships** between **similar time-steps**, which wouldn't be possible if we directly fed the single number to the model.

Thus, the **training phase** may be summarised as shown in the figure below:

1. Randomly select a time step & encode it



2. Add noise to image



$$x_t = \sqrt{\bar{a}_t} x_0 + \sqrt{1 - \bar{a}_t} \epsilon$$

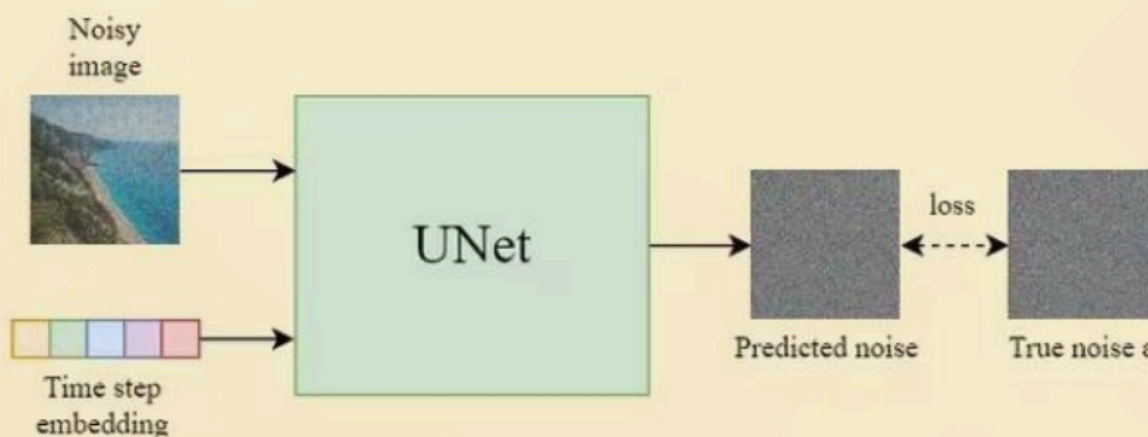
Adjust the amount of noise according to the time step t

$$\epsilon \sim \mathcal{N}(0, 1)$$

$$\alpha_t = 1 - \beta_t$$

$$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$$

3. Train the UNet



SAMPLING AND FINAL DETAILS

It is now worth discussing **how the model operates** once training is over.

Remember that in the introduction we said that the core idea behind diffusion models is that they can **take small steps** to **gradually produce a good sample**.

But wait:

*We've seen that the network is trained to predict the **entire noise** that was added to the original image, **not** to go **through each single diffusion step**; so what **role** does the **underlying process** play?*

The **diffusion process** is **indeed important**, particularly during **sampling**; that is, when we use the model to actually **generate images**.

The sampling process is performed as follows:

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 
```

SAMPLING

1. We **sample random Gaussian noise** from an **isotropic distribution** (recall that this assumption was made when we presented the *reverse diffusion process*).
2. We perform a **reverse diffusion process** of T steps as follows:
 1. Compute the **network's output** for the **previous noise level** ($\mathbf{x}_t$).
 2. **Subtract a small fraction** of the prediction from $\mathbf{x}_t$ to **produce a slightly less noisy image** $\mathbf{x}_{t-1}$.
 3. **Repeat** until $t = 1$.
3. At last, a **real image** is **obtained**.

This process is shown in the figure below:

1. Sample a Gaussian noise

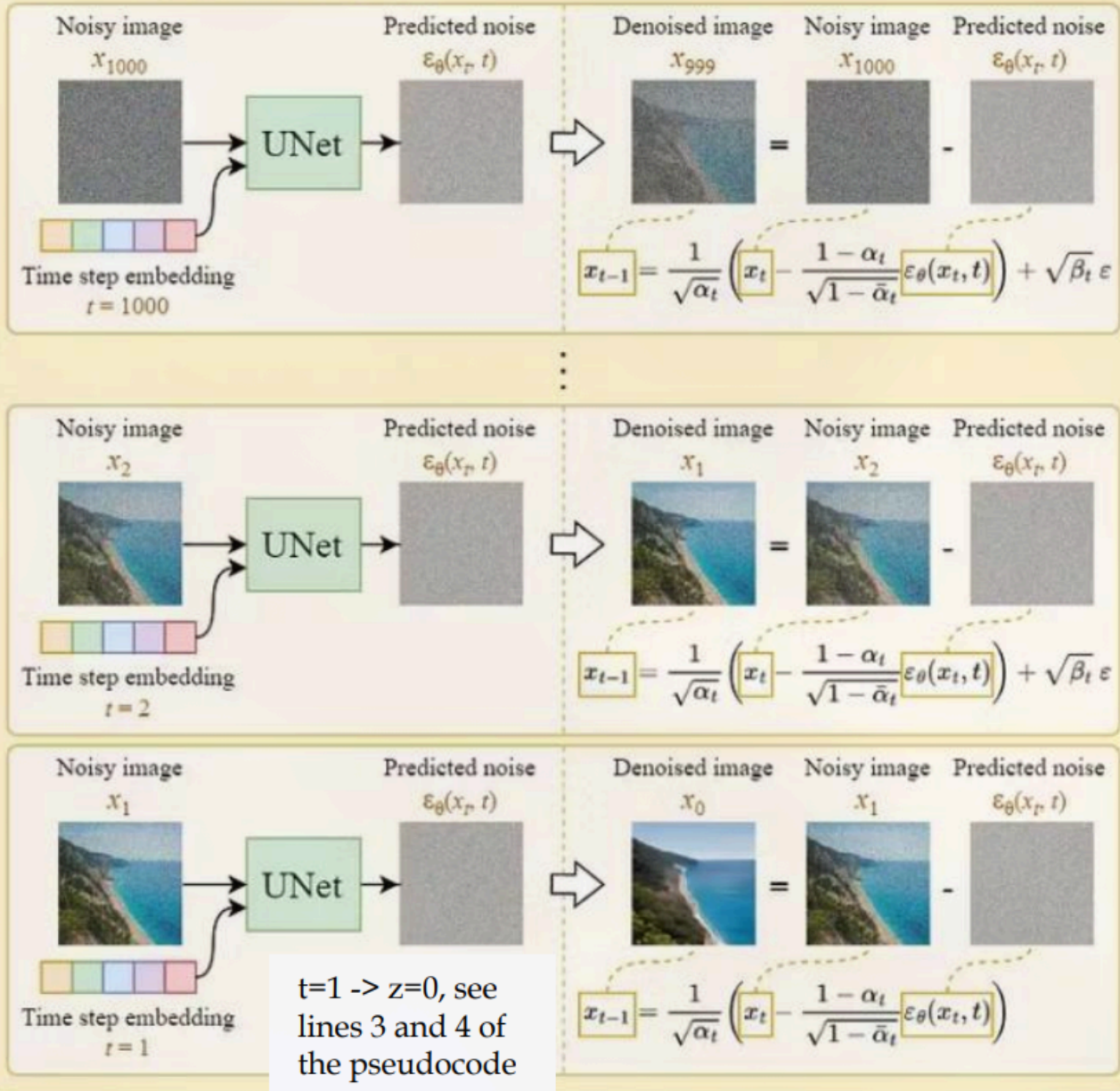
$$x_T \sim N(0, I)$$

E.g. $T = 1000$

$$x_{1000} \sim N(0, I)$$

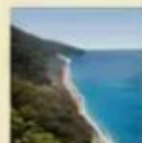


2. Iteratively denoise the image



3. Output the denoised image

Denoised image x_0



Immediately **subtracting the entire predicted noise** would produce highly **imperfect results**.

Instead, we **only subtract a tiny fraction** of noise from the starting image, then we **feed the result back into the network** and we **keep repeating** the process.

💡 INTUITION

As introduced at the *beginning of the paragraph* and also at > [REVERSE DIFFUSION](#), the idea is that **taking one small step at a time** allows the model to **correct itself**, follow a **more stable trajectory** and **produce better results**.

Because we have gone over *many concepts*, it may be useful to summarise them in simple terms:

📋 DIFFUSION MODELS - SUMMARY

1. **Forward diffusion** is used to **produce training examples** for the model.
2. **Training** is done by **comparing** the **predicted noise** to the noise that was **actually applied**.
3. **Inference** (*sampling*) consists in **mimicking** a **reverse diffusion process** that **gradually reduces noise** to **produce a real image**.